Chapter Contents

Chapter 20 - Second-Order Numerical Methods 4

Turning Order Into State 5

The Goal Of This Block 5

Why A Numerical Method Wants First-Order Form 5

The Reduction Pipeline 6

How The Initial Conditions Move 7

A Complete Reduction Example 7

Why The Highest-Derivative Coefficient Matters 9

A Third-Order Preview 9

Common Reduction Mistakes 10

Block 1 Summary 10

A Shared Benchmark Problem 11

The Goal Of This Block 11

The Second-Order Problem 11

Name The Two Right-Hand Sides 12

An Exact Benchmark For Checking 12

What One Numerical Row Must Store 13

Block 2 Summary 14

Euler's Method For A Pair Of States 14

The Goal Of This Block 14

The Paired Euler Formula 14

Why The Two Updates Are Simultaneous 15

Euler Step 0 To Step 1 15

Euler Step 1 To Step 2 17

Compare With The Exact State 18

Common Mistake: Feeding A New Value Into The Old Step 19

Block 3 Summary 19

Classical RK4 For A Coupled Pair 20

The Goal Of This Block 20

Two Increments At Every Stage 20

The Paired RK4 Update 21

RK4 Stage 1 For The Benchmark 22

RK4 Stage 2 23

RK4 Stage 3 24

RK4 Stage 4 . . . 25
Combine The Four Stage Pairs . . . 26
Check Both Components . . . 27
Euler And RK4 Across The Interval . . . 28
Common RK4 Pairing Mistakes . . . 29
Block 4 Summary . . . 29
ABM4 For Two Coupled Equations . . . 30
The Goal Of This Block . . . 30
Two Histories Must Be Stored . . . 30
The Paired AB4 Predictors . . . 31
The Predicted Endpoint Derivatives . . . 31
The Paired AM4 Correctors . . . 31
The RK4 Start-Up Table . . . 32
ABM Step 1: Predict Both State Components . . . 33
ABM Step 2: Evaluate The Predicted Derivatives . . . 34
ABM Step 3: Correct Both Components . . . 35
Compare Prediction, Correction, And Exact State . . . 36
Block 5 Summary . . . 36
Milne's Method For The System . . . 37
The Goal Of This Block . . . 37
The Paired Milne Predictors . . . 37
The Paired Milne Correctors . . . 37
A Complete Milne Prediction . . . 38
Calculate The Predicted Slopes And Correct . . . 39
Completed Milne State . . . 40
A Two-Component Gap Diagnostic . . . 40
Block 6 Summary . . . 40
Generalisation, Diagnostics, And Workflow . . . 41
The Goal Of This Block . . . 41
The Vector View . . . 41
Higher-Order Scalar Equations . . . 42
Accuracy And Stability Are System Properties . . . 42
Common System-Level Mistakes . . . 43
A Reliable Calculation Workflow . . . 43
Block 7 Summary . . . 44
Original Practice Set . . . 44

How To Use These Problems 44
Problems 1 To 4: Reduction And Euler 44
Problems 5 To 8: Updates And RK4 45
Problems 9 To 12: Multistep Methods And Diagnostics 46
Worked Answers: Problems 1 To 4 47
Worked Answers: Problems 5 To 8 50
Worked Answers: Problems 9 To 12 53
Chapter 20 Final Summary 56

Chapter 20 - Second-Order Numerical Methods

Chapter 17 showed that a higher-order differential equation can be rewritten as a first-order system. Chapters 18 and 19 developed numerical methods for first-order equations.

This chapter joins those ideas:

> **rewrite one second-order equation as two linked first-order equations, then advance both unknowns together.**

The two unknowns will usually be:

- $y(x)$, the position-like state
- $v(x) = y'(x)$, the velocity-like state

We will adapt Euler, classical RK4, Adams–Bashforth–Moulton, and Milne's method to this paired system. The figures, examples, numerical data, and exercises are independently constructed. The reference text is used only as a map of the standard methods.

Turning Order Into State

The Goal Of This Block

The goal is to convert a second-order initial-value problem into a first-order system without losing an equation or an initial condition.

Why A Numerical Method Wants First-Order Form

Suppose the differential equation can be solved for its highest derivative:

$$y'' = F(x, y, y').$$

A first-order numerical method expects a rule that gives the derivative of each current state variable. Introduce:

$$v = y'.$$

Differentiate this definition:

$$v' = y''.$$

Now substitute v for y' and v' for y'' in the original equation:

$$v' = F(x, y, v).$$

The equivalent first-order system is therefore:

$$\boxed{\begin{aligned} y' &= v, \\ v' &= F(x, y, v). \end{aligned}}$$

This is not an approximation. It is a change of representation.

The Reduction Pipeline

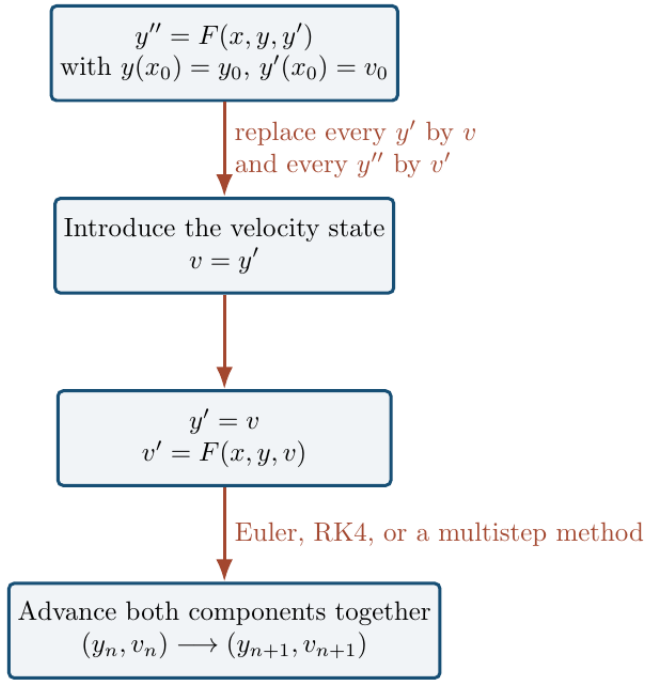

Figure 1: Pipeline from a second-order equation to a first-order numerical system

The diagram separates two jobs:

1. **Model conversion:** define $v = y'$ and obtain equations for y' and v'.
2. **Numerical advancement:** apply one chosen method to both equations at the same stage points.

The reduction changes the object that the numerical method advances:

the numerical state is no longer one number y_n; it is the pair (y_n, v_n).

How The Initial Conditions Move

For a second-order initial-value problem:

$$y(x_0) = y_0, \qquad y'(x_0) = v_0,$$

the substitution $v = y'$ turns the conditions into:

$$\boxed{y(x_0) = y_0, \qquad v(x_0) = v_0}.$$

Both values are needed. The first gives the initial position-like state; the second gives the initial velocity-like state.

Knowing only $y(x_0)$ does not normally determine a unique second-order solution.

A Complete Reduction Example

Convert:

$$2y'' + (1 + x)y' - 3y = x^2, \qquad y(1) = 2, \qquad y'(1) = -1.$$

First isolate the highest derivative. Subtract $(1 + x)y'$ and add $3y$ on both sides:

$$2y'' = x^2 - (1 + x)y' + 3y.$$

Starting from $2y'' = x^2 - (1+x)y' + 3y$, divide every term by 2:

$$y'' = \frac{x^2 - (1+x)y' + 3y}{2}.$$

Define:

$$v = y'.$$

Then:

$$y' = v$$

Differentiate the new state variable to obtain the second system equation:

$$\begin{aligned} v' &= y'' \\ &= \frac{x^2 - (1+x)v + 3y}{2}. \end{aligned}$$

Transfer the initial conditions one at a time:

$$y(1) = 2,$$

and, because $v = y'$:

$$v(1) = y'(1) = -1.$$

The first-order initial-value system is:

$$\boxed{\begin{aligned} y' &= v, \\ v' &= \frac{x^2 - (1+x)v + 3y}{2}, \\ y(1) &= 2, \\ v(1) &= -1. \end{aligned}}$$

Why The Highest-Derivative Coefficient Matters

If the equation begins as:

$$a(x,y,y')y'' = R(x,y,y'),$$

then solving for y'' requires division by a:

$$y'' = \frac{R(x,y,y')}{a(x,y,y')}.$$

The reduction is valid only where:

$$a(x,y,y') \neq 0.$$

If this coefficient vanishes, the first-order system becomes singular there. A numerical routine should not silently step through such a point.

A Third-Order Preview

For:

$$u''' = xu'' - 2u' + u,$$

define one state for each derivative below the highest order:

$$y_1 = u, \qquad y_2 = u', \qquad y_3 = u''.$$

Differentiate each definition:

$$\begin{aligned} y_1' &= u' = y_2, \\ y_2' &= u'' = y_3, \\ y_3' &= u''' = xy_3 - 2y_2 + y_1. \end{aligned}$$

Thus an order-m scalar equation normally becomes a system with m state components.

Common Reduction Mistakes

Watch for four common errors:

- writing $v' = y'$ instead of $v' = y''$
- replacing y'' by v rather than by v'
- leaving an old y' inside the second system equation
- forgetting to convert $y'(x_0)$ into the initial value $v(x_0)$

The reliable check is to substitute $v = y'$ and $v' = y''$ back into the system. The original equation and both initial conditions should reappear.

Block 1 Summary

Solve the equation for y'', define $v = y'$, and write:

$$y' = v, \qquad v' = F(x, y, v).$$

The numerical state is the pair (y, v), and both initial values must be carried into the system.

A Shared Benchmark Problem

The Goal Of This Block

The goal is to establish one independently chosen problem that can be used to compare every numerical method in this chapter.

The Second-Order Problem

We will use:

$$y'' + 2y' + 2y = 2x, \qquad y(0) = 1, \qquad y'(0) = 0.$$

Isolate y'' by subtracting $2y'$ and $2y$:

$$y'' = 2x - 2y' - 2y.$$

Define:

$$v = y'.$$

The first-order system is:

$$\boxed{\begin{aligned} y' &= v, \\ v' &= 2x - 2v - 2y, \\ y(0) &= 1, \\ v(0) &= 0. \end{aligned}}$$

Name The Two Right-Hand Sides

Write a general two-component system as:

$$\begin{aligned} y' &= f(x, y, v), \\ v' &= g(x, y, v). \end{aligned}$$

For the benchmark problem:

$$\boxed{\begin{aligned} f(x, y, v) &= v, \\ g(x, y, v) &= 2x - 2v - 2y. \end{aligned}}$$

The symbols f and g describe different derivatives. Here f produces the position derivative, while g produces the velocity derivative.

An Exact Benchmark For Checking

The exact solution is:

$$y(x) = x - 1 + e^{-x}(2\cos x + \sin x).$$

Differentiate the product $e^{-x}(2\cos x + \sin x)$ explicitly:

$$\begin{aligned} y'(x) &= 1 - e^{-x}(2\cos x + \sin x) + e^{-x}(-2\sin x + \cos x) \\ &= 1 + e^{-x}(-\cos x - 3\sin x). \end{aligned}$$

Therefore the exact velocity is:

$$v(x) = 1 - e^{-x}(\cos x + 3\sin x).$$

Using the displayed exact formulas for $y(x)$ and $v(x)$, check the initial values locally:

$$\begin{aligned} y(0) &= 0 - 1 + 1(2 + 0) \\ &= 1, \end{aligned}$$

Check the velocity initial condition separately:

$$\begin{aligned} v(0) &= 1 - 1(1 + 0) \\ &= 0. \end{aligned}$$

The exact formula is used only to assess numerical error. The numerical methods themselves do not use it.

What One Numerical Row Must Store

At each grid point x_n, store:

$$(x_n, y_n, v_n).$$

For multistep work, also store both derivatives:

$$f_n = f(x_n, y_n, v_n), \qquad g_n = g(x_n, y_n, v_n).$$

For this problem:

$$f_n = v_n, \qquad g_n = 2x_n - 2v_n - 2y_n.$$

One row therefore describes the entire approximate state, not just y_n.

Block 2 Summary

The benchmark system is:

$$y' = v, \qquad v' = 2x - 2v - 2y.$$

Every method must advance y and v together. The exact solution provides a check but does not enter the numerical update.

Euler's Method For A Pair Of States

The Goal Of This Block

The goal is to apply explicit Euler to both system equations and understand why both updates use the same old state.

The Paired Euler Formula

For:

$$y' = f(x, y, v), \qquad v' = g(x, y, v),$$

calculate both old-state derivatives:

$$f_n = f(x_n, y_n, v_n), \qquad g_n = g(x_n, y_n, v_n).$$

Then update:

$$\boxed{\begin{aligned} y_{n+1} &= y_n + hf_n, \\ v_{n+1} &= v_n + hg_n, \\ x_{n+1} &= x_n + h. \end{aligned}}$$

For any reduced second-order equation, $f_n = v_n$, so the position update is:

$$y_{n+1} = y_n + hv_n.$$

Why The Two Updates Are Simultaneous

Explicit Euler uses the derivative vector at one point:

$$(x_n, y_n, v_n).$$

Therefore both derivatives must be evaluated before either new component is used:

$$\begin{aligned} f_n &= f(x_n, y_n, v_n), \\ g_n &= g(x_n, y_n, v_n). \end{aligned}$$

Only after calculating both f_n and g_n do we form y_{n+1} and v_{n+1}.

Using a newly updated y_{n+1} inside the formula for v_{n+1} would define a different method.

Euler Step 0 To Step 1

Use the benchmark system with:

$$h = 0.1, \qquad (x_0, y_0, v_0) = (0, 1, 0).$$

Calculate the two old-state derivatives:

$$\begin{aligned} f_0 &= v_0 \\ &= 0, \end{aligned}$$

Calculate the velocity derivative at the same old state:

$$\begin{aligned} g_0 &= 2x_0 - 2v_0 - 2y_0 \\ &= 2(0) - 2(0) - 2(1) \\ &= -2. \end{aligned}$$

Update the position component:

$$\begin{aligned} y_1 &= y_0 + hf_0 \\ &= 1 + 0.1(0) \\ &= 1. \end{aligned}$$

Update the velocity component using the already calculated old slope g_0:

$$\begin{aligned} v_1 &= v_0 + hg_0 \\ &= 0 + 0.1(-2) \\ &= -0.2. \end{aligned}$$

Update the independent variable:

$$x_1 = x_0 + h = 0.1.$$

The first Euler state is:

$$\boxed{(x_1, y_1, v_1) = (0.1, 1, -0.2)}.$$

Euler Step 1 To Step 2

Begin from the entire corrected row:

$$(x_1, y_1, v_1) = (0.1, 1, -0.2).$$

The position derivative is:

$$\begin{aligned} f_1 &= v_1 \\ &= -0.2. \end{aligned}$$

The velocity derivative is:

$$\begin{aligned} g_1 &= 2x_1 - 2v_1 - 2y_1 \\ &= 2(0.1) - 2(-0.2) - 2(1) \\ &= 0.2 + 0.4 - 2 \\ &= -1.4. \end{aligned}$$

Use these two derivatives in the paired update:

$$\begin{aligned} y_2 &= y_1 + hf_1 \\ &= 1 + 0.1(-0.2) \\ &= 0.98, \end{aligned}$$

Update the velocity component with its matching derivative:

$$\begin{aligned} v_2 &= v_1 + hg_1 \\ &= -0.2 + 0.1(-1.4) \\ &= -0.34. \end{aligned}$$

Update the independent variable to complete the second Euler row:

$$x_2 = 0.1 + 0.1 = 0.2.$$

The completed second Euler row is:

$$\boxed{(x_2, y_2, v_2) = (0.2, 0.98, -0.34)}.$$

Compare With The Exact State

At $x = 0.2$, the exact values are approximately:

$$y(0.2) \approx 0.967477986, \qquad v(0.2) \approx -0.290380720.$$

The Euler errors in both state components are:

$$\begin{aligned} |y_2 - y(0.2)| &= |0.98 - 0.967477986| \\ &\approx 0.012522014, \end{aligned}$$

The velocity-component error is:

$$\begin{aligned} |v_2 - v(0.2)| &= |-0.34 - (-0.290380720)| \\ &\approx 0.049619280. \end{aligned}$$

An acceptable position value does not guarantee an equally accurate velocity value. Both components require inspection.

Common Mistake: Feeding A New Value Into The Old Step

The explicit Euler velocity update is:

$$v_{n+1} = v_n + hg(x_n, y_n, v_n).$$

It is not:

$$v_{n+1} = v_n + hg(x_n, y_{n+1}, v_n).$$

The second expression mixes an old input and velocity with a new position. Unless a specifically designed semi-implicit method is intended, that mixture is an indexing error.

Block 3 Summary

System Euler evaluates both derivatives at the same old state and then updates both components:

$$(y_n, v_n) \longrightarrow (y_{n+1}, v_{n+1}).$$

Calculate both old slopes first. Never let the first assignment silently change the input used by the second assignment.

Classical RK4 For A Coupled Pair

The Goal Of This Block

The goal is to build four linked stage pairs and use each pair consistently in both system equations.

Two Increments At Every Stage

For the system:

$$y' = f(x, y, v), \qquad v' = g(x, y, v),$$

define:

- K_j as the stage increment for y
- L_j as the stage increment for v

Both already include the step size h.

Stage 1 is:

$$\boxed{\begin{aligned} K_1 &= hf(x_n, y_n, v_n), \\ L_1 &= hg(x_n, y_n, v_n). \end{aligned}}$$

Stage 2 uses one shared midpoint state:

$$\boxed{\begin{aligned} K_2 &= hf\left(x_n + \frac{h}{2}, y_n + \frac{K_1}{2}, v_n + \frac{L_1}{2}\right), \\ L_2 &= hg\left(x_n + \frac{h}{2}, y_n + \frac{K_1}{2}, v_n + \frac{L_1}{2}\right). \end{aligned}}$$

Stage 3 uses the second stage pair:

$$\boxed{\begin{aligned} K_3 &= hf\left(x_n + \frac{h}{2}, y_n + \frac{K_2}{2}, v_n + \frac{L_2}{2}\right), \\ L_3 &= hg\left(x_n + \frac{h}{2}, y_n + \frac{K_2}{2}, v_n + \frac{L_2}{2}\right). \end{aligned}}$$

Stage 4 uses the third stage pair:

$$\boxed{\begin{aligned} K_4 &= hf(x_n + h, y_n + K_3, v_n + L_3), \\ L_4 &= hg(x_n + h, y_n + K_3, v_n + L_3). \end{aligned}}$$

The Paired RK4 Update

Combine the K increments only to update y:

$$\boxed{y_{n+1} = y_n + \frac{K_1 + 2K_2 + 2K_3 + K_4}{6}}.$$

Combine the L increments only to update v:

$$\boxed{v_{n+1} = v_n + \frac{L_1 + 2L_2 + 2L_3 + L_4}{6}}.$$

Finally:

$$x_{n+1} = x_n + h.$$

The weights match, but the two increment sequences are not interchangeable.

RK4 Stage 1 For The Benchmark

Use:

$$(x_0, y_0, v_0) = (0, 1, 0), \qquad h = 0.1.$$

Recall:

$$f(x, y, v) = v, \qquad g(x, y, v) = 2x - 2v - 2y.$$

Calculate the position increment:

$$\begin{aligned} K_1 &= 0.1 f(0, 1, 0) \\ &= 0.1(0) \\ &= 0. \end{aligned}$$

Calculate the velocity increment from the same state:

$$\begin{aligned} L_1 &= 0.1 g(0, 1, 0) \\ &= 0.1(2(0) - 2(0) - 2(1)) \\ &= -0.2. \end{aligned}$$

RK4 Stage 2

The stage-2 input is:

$$\begin{aligned} x_0 + \frac{h}{2} &= 0.05, \\ y_0 + \frac{K_1}{2} &= 1 + \frac{0}{2} = 1, \\ v_0 + \frac{L_1}{2} &= 0 + \frac{-0.2}{2} = -0.1. \end{aligned}$$

Thus the shared midpoint state is $(0.05, 1, -0.1)$.

At the shared stage-2 state $(0.05, 1, -0.1)$, calculate the position increment:

$$\begin{aligned} K_2 &= 0.1f(0.05, 1, -0.1) \\ &= 0.1(-0.1) \\ &= -0.01, \end{aligned}$$

At the same stage-2 state, calculate the velocity increment:

$$\begin{aligned} L_2 &= 0.1g(0.05, 1, -0.1) \\ &= 0.1(2(0.05) - 2(-0.1) - 2(1)) \\ &= 0.1(0.1 + 0.2 - 2) \\ &= -0.17. \end{aligned}$$

RK4 Stage 3

Stage 3 must use K_2 and L_2, not the first-stage pair:

$$\begin{aligned} x_0 + \frac{h}{2} &= 0.05, \\ y_0 + \frac{K_2}{2} &= 1 + \frac{-0.01}{2} = 0.995, \\ v_0 + \frac{L_2}{2} &= 0 + \frac{-0.17}{2} = -0.085. \end{aligned}$$

At $(0.05, 0.995, -0.085)$:

$$\begin{aligned} K_3 &= 0.1f(0.05, 0.995, -0.085) \\ &= 0.1(-0.085) \\ &= -0.0085, \end{aligned}$$

At the same stage-3 state, calculate the velocity increment:

$$\begin{aligned} L_3 &= 0.1g(0.05, 0.995, -0.085) \\ &= 0.1(0.1 + 0.17 - 1.99) \\ &= -0.172. \end{aligned}$$

RK4 Stage 4

Stage 4 uses the full third-stage increments:

$$\begin{aligned} x_0 + h &= 0.1, \\ y_0 + K_3 &= 1 - 0.0085 = 0.9915, \\ v_0 + L_3 &= 0 - 0.172 = -0.172. \end{aligned}$$

At $(0.1, 0.9915, -0.172)$:

$$\begin{aligned} K_4 &= 0.1 f(0.1, 0.9915, -0.172) \\ &= 0.1(-0.172) \\ &= -0.0172, \end{aligned}$$

At the same stage-4 state, calculate the velocity increment:

$$\begin{aligned} L_4 &= 0.1 g(0.1, 0.9915, -0.172) \\ &= 0.1(0.2 + 0.344 - 1.983) \\ &= -0.1439. \end{aligned}$$

Combine The Four Stage Pairs

Use the K sequence to update y:

$$\begin{aligned} y_1 &= y_0 + \frac{K_1 + 2K_2 + 2K_3 + K_4}{6} \\ &= 1 + \frac{0 + 2(-0.01) + 2(-0.0085) - 0.0172}{6} \\ &= 1 + \frac{-0.0542}{6} \\ &\approx 0.990966667. \end{aligned}$$

Use the L sequence to update v:

$$\begin{aligned} v_1 &= v_0 + \frac{L_1 + 2L_2 + 2L_3 + L_4}{6} \\ &= 0 + \frac{-0.2 + 2(-0.17) + 2(-0.172) - 0.1439}{6} \\ &= \frac{-1.0279}{6} \\ &\approx -0.171316667. \end{aligned}$$

The one-step RK4 state is:

$$\boxed{(x_1, y_1, v_1) \approx (0.1, 0.990966667, -0.171316667)}.$$

Check Both Components

The exact state at $x = 0.1$ is approximately:

$$(y(0.1), v(0.1)) \approx (0.990967011, -0.171316033).$$

The absolute position error is:

$$|0.990966667 - 0.990967011| \approx 3.44 \times 10^{-7}.$$

The absolute velocity error is:

$$|-0.171316667 - (-0.171316033)| \approx 6.34 \times 10^{-7}.$$

Both components have the expected high one-step accuracy.

Euler And RK4 Across The Interval

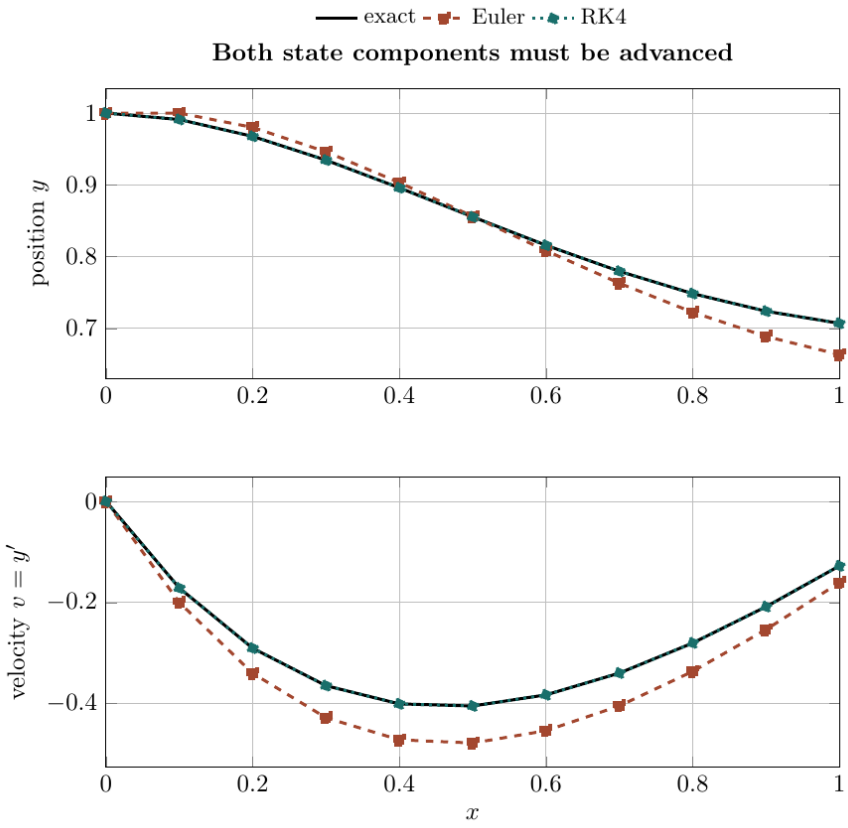

Figure 2: Position and velocity approximations from Euler and RK4

The upper panel compares position y. The lower panel compares velocity $v = y'$. With $h = 0.1$, RK4 remains visually close to the exact state in both panels, while Euler's error accumulates more noticeably.

The practical meaning:

> **a system method succeeds only when the entire state vector is advanced accurately; plotting only y can hide a much larger error in v.**

Common RK4 Pairing Mistakes

Do not:

- use K_1 with L_2 to build one midpoint state
- use K_2 and L_1 in stage 3
- use a K increment in the update for v
- forget that every K_j and L_j already contains h

Each stage is a matched pair:

$$(K_1, L_1), \quad (K_2, L_2), \quad (K_3, L_3), \quad (K_4, L_4).$$

Block 4 Summary

System RK4 samples four shared stage states. Every stage produces one increment for each component, and the two weighted sums are formed separately.

The stage coordinates matter as much as the final $1, 2, 2, 1$ weights.

ABM4 For Two Coupled Equations

The Goal Of This Block

The goal is to apply the four-step Adams–Bashforth predictor and Adams–Moulton corrector to both state components without crossing their derivative histories.

Two Histories Must Be Stored

For:

$$y' = f(x, y, v), \qquad v' = g(x, y, v),$$

store:

$$f_j = f(x_j, y_j, v_j)$$

Store the velocity derivative history separately:

$$g_j = g(x_j, y_j, v_j).$$

The f history advances y. The g history advances v.

For the benchmark problem:

$$f_j = v_j, \qquad g_j = 2x_j - 2v_j - 2y_j.$$

The Paired AB4 Predictors

Use the standard AB4 weights separately for each derivative history:

$$\boxed{\begin{aligned} y_{n+1}^{(p)} &= y_n + \frac{h}{24}\left(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right), \\ v_{n+1}^{(p)} &= v_n + \frac{h}{24}\left(55g_n - 59g_{n-1} + 37g_{n-2} - 9g_{n-3}\right). \end{aligned}}$$

The two predictors use the same coefficients but generally produce different numbers.

The Predicted Endpoint Derivatives

After predicting both state components, evaluate both right-hand sides at one predicted endpoint:

$$\boxed{\begin{aligned} f_{n+1}^{(p)} &= f\left(x_{n+1}, y_{n+1}^{(p)}, v_{n+1}^{(p)}\right), \\ g_{n+1}^{(p)} &= g\left(x_{n+1}, y_{n+1}^{(p)}, v_{n+1}^{(p)}\right). \end{aligned}}$$

Both predicted state values belong in both function calls when the system is fully coupled.

The Paired AM4 Correctors

Correct each component with its matching derivative sequence:

$$\boxed{\begin{aligned} y_{n+1} &= y_n + \frac{h}{24}\left(9f_{n+1}^{(p)} + 19f_n - 5f_{n-1} + f_{n-2}\right), \\ v_{n+1} &= v_n + \frac{h}{24}\left(9g_{n+1}^{(p)} + 19g_n - 5g_{n-1} + g_{n-2}\right). \end{aligned}}$$

Do not correct y and then use that corrected y in the displayed one-correction formula for v. Both corrections use the same predicted endpoint derivatives.

The RK4 Start-Up Table

ABM4 needs four complete state rows. RK4 with $h = 0.1$ supplies:

j	x_j	y_j	$v_j = f_j$	$g_j = 2x_j - 2v_j - 2y_j$
0	0.0	1.000000000	0.000000000	−2.000000000
1	0.1	0.990966667	−0.171316667	−1.439300000
2	0.2	0.967477189	−0.290381623	−0.954191134
3	0.3	0.934386804	−0.364511820	−0.539749969

For example, calculate the final listed g value from its own row:

$$\begin{aligned} g_3 &= 2x_3 - 2v_3 - 2y_3 \\ &= 2(0.3) - 2(-0.364511820) - 2(0.934386804) \\ &\approx -0.539749969. \end{aligned}$$

The first multistep target is:

$$x_4 = x_3 + h = 0.4.$$

ABM Step 1: Predict Both State Components

For the position predictor, form the f bracket:

$$\begin{aligned} B_f &= 55f_3 - 59f_2 + 37f_1 - 9f_0 \\ &= 55(-0.364511820) - 59(-0.290381623) \\ &\quad + 37(-0.171316667) - 9(0) \\ &\approx -9.254351044. \end{aligned}$$

Substitute this bracket into the AB4 position predictor:

$$\begin{aligned} y_4^{(p)} &= y_3 + \frac{h}{24} B_f \\ &= 0.934386804 + \frac{0.1}{24}(-9.254351044) \\ &\approx 0.895827008. \end{aligned}$$

For the velocity predictor, form the separate g bracket:

$$\begin{aligned} B_g &= 55g_3 - 59g_2 + 37g_1 - 9g_0 \\ &= 55(-0.539749969) - 59(-0.954191134) \\ &\quad + 37(-1.439300000) - 9(-2) \\ &\approx -8.643071384. \end{aligned}$$

Substitute this bracket into the AB4 velocity predictor:

$$\begin{aligned} v_4^{(p)} &= v_3 + \frac{h}{24} B_g \\ &= -0.364511820 + \frac{0.1}{24}(-8.643071384) \\ &\approx -0.400524618. \end{aligned}$$

Completed AB4 Prediction. The completed AB4 prediction gives:

$$\boxed{(y_4^{(p)}, v_4^{(p)}) \approx (0.895827008, -0.400524618)}.$$

ABM Step 2: Evaluate The Predicted Derivatives

For the benchmark system, $f = v$. Therefore:

$$\begin{aligned} f_4^{(p)} &= f(0.4, y_4^{(p)}, v_4^{(p)}) \\ &= v_4^{(p)} \\ &\approx -0.400524618. \end{aligned}$$

Evaluate g using both predicted state components:

$$\begin{aligned} g_4^{(p)} &= 2x_4 - 2v_4^{(p)} - 2y_4^{(p)} \\ &= 2(0.4) - 2(-0.400524618) - 2(0.895827008) \\ &\approx -0.190604782. \end{aligned}$$

ABM Step 3: Correct Both Components

Form the AM4 position bracket:

$$\begin{aligned} C_f &= 9f_4^{(p)} + 19f_3 - 5f_2 + f_1 \\ &= 9(-0.400524618) + 19(-0.364511820) \\ &\qquad - 5(-0.290381623) - 0.171316667 \\ &\approx -9.249854694. \end{aligned}$$

Correct y_4:

$$\begin{aligned} y_4 &= y_3 + \frac{h}{24} C_f \\ &= 0.934386804 + \frac{0.1}{24}(-9.249854694) \\ &\approx 0.895845743. \end{aligned}$$

Form the AM4 velocity bracket:

$$\begin{aligned} C_g &= 9g_4^{(p)} + 19g_3 - 5g_2 + g_1 \\ &= 9(-0.190604782) + 19(-0.539749969) \\ &\qquad - 5(-0.954191134) - 1.439300000 \\ &\approx -8.639036775. \end{aligned}$$

Correct v_4:

$$\begin{aligned} v_4 &= v_3 + \frac{h}{24} C_g \\ &= -0.364511820 + \frac{0.1}{24}(-8.639036775) \\ &\approx -0.400507807. \end{aligned}$$

Completed ABM4 State. The completed ABM4 correction gives the state:

$$\boxed{(y_4, v_4) \approx (0.895845743, -0.400507807)}.$$

Compare Prediction, Correction, And Exact State

At $x = 0.4$:

$$\begin{aligned}(y_4^{(p)}, v_4^{(p)}) &\approx (0.895827008, -0.400524618),\\ (y_4, v_4) &\approx (0.895845743, -0.400507807),\\ (y(0.4), v(0.4)) &\approx (0.895846217, -0.400510411).\end{aligned}$$

The correction improves the position prediction substantially. The velocity also changes, but a correction is not guaranteed to move every component monotonically closer on every single step.

Block 5 Summary

ABM4 carries two parallel derivative histories. Predict both components, evaluate both derivatives at one predicted endpoint, and then correct both components with their matching histories.

Milne's Method For The System

The Goal Of This Block

The goal is to apply Milne's predictor and Simpson-type corrector to both components and keep the unusual base indices visible.

The Paired Milne Predictors

For each derivative history:

$$\boxed{\begin{aligned} y_{n+1}^{(p)} &= y_{n-3} + \frac{4h}{3}\left(2f_n - f_{n-1} + 2f_{n-2}\right), \\ v_{n+1}^{(p)} &= v_{n-3} + \frac{4h}{3}\left(2g_n - g_{n-1} + 2g_{n-2}\right). \end{aligned}}$$

The predictor base values are y_{n-3} and v_{n-3}. They are not y_n and v_n.

The Paired Milne Correctors

After calculating $f_{n+1}^{(p)}$ and $g_{n+1}^{(p)}$, use:

$$\boxed{\begin{aligned} y_{n+1} &= y_{n-1} + \frac{h}{3}\left(f_{n-1} + 4f_n + f_{n+1}^{(p)}\right), \\ v_{n+1} &= v_{n-1} + \frac{h}{3}\left(g_{n-1} + 4g_n + g_{n+1}^{(p)}\right). \end{aligned}}$$

The corrector base index is $n-1$, so it differs from both the predictor base and the newest stored row.

A Complete Milne Prediction

Use the benchmark start-up table and set $n = 3$ to target $x_4 = 0.4$.

Form the position bracket:

$$\begin{aligned} M_f &= 2f_3 - f_2 + 2f_1 \\ &= 2(-0.364511820) - (-0.290381623) + 2(-0.171316667) \\ &\approx -0.781275351. \end{aligned}$$

The predictor base is $y_0 = 1$:

$$\begin{aligned} y_4^{(p)} &= y_0 + \frac{4h}{3} M_f \\ &= 1 + \frac{4(0.1)}{3}(-0.781275351) \\ &\approx 0.895829953. \end{aligned}$$

Form the velocity bracket:

$$\begin{aligned} M_g &= 2g_3 - g_2 + 2g_1 \\ &= 2(-0.539749969) - (-0.954191134) + 2(-1.439300000) \\ &\approx -3.003908804. \end{aligned}$$

The velocity predictor base is $v_0 = 0$:

$$\begin{aligned} v_4^{(p)} &= v_0 + \frac{4h}{3} M_g \\ &= 0 + \frac{4(0.1)}{3}(-3.003908804) \\ &\approx -0.400521174. \end{aligned}$$

Calculate The Predicted Slopes And Correct

At the predicted state:

$$\begin{aligned} f_4^{(p)} &= v_4^{(p)} \\ &\approx -0.400521174, \end{aligned}$$

At the same predicted state, calculate the velocity derivative:

$$\begin{aligned} g_4^{(p)} &= 2(0.4) - 2v_4^{(p)} - 2y_4^{(p)} \\ &= 0.8 - 2(-0.400521174) - 2(0.895829953) \\ &\approx -0.190617559. \end{aligned}$$

Use the position corrector with base value y_2:

$$\begin{aligned} y_4 &= y_2 + \frac{h}{3}(f_2 + 4f_3 + f_4^{(p)}) \\ &= 0.967477189 + \frac{0.1}{3}(-0.290381623 + 4(-0.364511820) - 0.400521174) \\ &\approx 0.895845520. \end{aligned}$$

Use the velocity corrector with base value v_2:

$$\begin{aligned} v_4 &= v_2 + \frac{h}{3}(g_2 + 4g_3 + g_4^{(p)}) \\ &= -0.290381623 + \frac{0.1}{3}(-0.954191134 + 4(-0.539749969) - 0.190617559) \\ &\approx -0.400508575. \end{aligned}$$

Completed Milne State

The completed Milne correction gives the state:

$$\boxed{(y_4, v_4) \approx (0.895845520, -0.400508575)}.$$

A Two-Component Gap Diagnostic

The predictor–corrector changes are:

$$\begin{aligned} |y_4 - y_4^{(p)}| &\approx |0.895845520 - 0.895829953| \\ &\approx 1.56 \times 10^{-5}, \\ |v_4 - v_4^{(p)}| &\approx |-0.400508575 - (-0.400521174)| \\ &\approx 1.26 \times 10^{-5}. \end{aligned}$$

Check both gaps. A small change in y can coexist with a large change in v, especially in rapidly varying or stiff systems.

Block 6 Summary

Milne's method uses the same coefficients for both state components but different derivative histories. Keep $n-3$ visible in the predictor and $n-1$ visible in the corrector.

Generalisation, Diagnostics, And Workflow

The Goal Of This Block

The goal is to connect the component formulas to vector notation and build a reliable workflow for larger systems.

The Vector View

Collect the state variables into:

$$\mathbf{Y} = \begin{bmatrix} y \\ v \end{bmatrix}.$$

Collect their derivative rules into:

$$\mathbf{F}(x, \mathbf{Y}) = \begin{bmatrix} f(x, y, v) \\ g(x, y, v) \end{bmatrix}.$$

Then the system is:

$$\mathbf{Y}' = \mathbf{F}(x, \mathbf{Y}).$$

Euler becomes one vector equation:

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\mathbf{F}(x_n, \mathbf{Y}_n).$$

The compact notation does not remove any calculations. It states that the same method is applied component by component at shared stage states.

Higher-Order Scalar Equations

For an mth-order equation solved for $u^{(m)}$, define:

$$y_1 = u, \quad y_2 = u', \quad \dots, \quad y_m = u^{(m-1)}.$$

The first $m - 1$ equations form a derivative chain:

$$y_1' = y_2, \quad y_2' = y_3, \quad \dots, \quad y_{m-1}' = y_m.$$

The final equation comes from the original differential equation:

$$y_m' = u^{(m)}.$$

Euler, RK4, ABM4, and Milne then advance all m components.

Accuracy And Stability Are System Properties

For a smooth, nonstiff problem in a stable regime, the familiar global orders remain:

Method	Typical global order
Euler	1
Classical RK4	4
ABM4 predictor–corrector	4
Milne predictor–corrector	4

However, converting to a system does not make a difficult problem easy. Rapidly decaying modes, widely separated time scales, singular coefficients, or unstable dynamics can force a smaller step or a different numerical method.

Common System-Level Mistakes

Before trusting a table, check for:

- missing initial values after reduction
- unequal grid spacing in a fixed-step multistep formula
- f values used in the v update or g values used in the y update
- stage components taken from different RK4 stages
- a predicted y paired with an old v when evaluating a coupled derivative
- premature rounding of stored state and derivative histories
- a highest-derivative coefficient that vanishes on the interval

A Reliable Calculation Workflow

For a second-order numerical problem:

1. isolate y''
2. define $v = y'$
3. write and label $f(x, y, v)$ and $g(x, y, v)$
4. transfer both initial conditions
5. choose h and verify the grid
6. calculate every component of one stage before advancing
7. store complete rows $(x_n, y_n, v_n, f_n, g_n)$
8. for a multistep method, generate three additional rows with RK4
9. compare predictor and corrector gaps component by component
10. repeat with a smaller h or compare with a known solution when possible

Block 7 Summary

The formulas scale naturally to larger state vectors. The main discipline is not new algebra; it is preserving shared stage states, component identities, indices, and complete numerical history.

Original Practice Set

How To Use These Problems

Work with a small table containing every state component. Before substituting numbers, write the two derivative functions and label which history advances which component.

Problems 1 To 4: Reduction And Euler

Problem 1: Reduce A Variable-Coefficient Equation. Convert:

$$y'' + 3xy' - 2y = e^x, \qquad y(0) = 2, \qquad y'(0) = -1$$

to a first-order initial-value system using $v = y'$.

Problem 2: Detect A Singular Reduction. Consider:

$$xy'' + (1 + x)y' + y = 0.$$

Solve for y'' and explain why an initial condition at $x_0 = 0$ creates a problem for this standard-form reduction.

Problem 3: Reduce A Third-Order Equation. Convert:

$$u''' + u'' - xu' = \cos x$$

to a three-component first-order system.

Problem 4: Take One Euler Step. For:

$$y'' = x - y - 2y', \qquad y(0) = 0, \qquad y'(0) = 1,$$

use Euler with $h = 0.2$ to calculate (y_1, v_1).

Problems 5 To 8: Updates And RK4

Problem 5: Take A Second Euler Step. Continue Problem 4 from $x_1 = 0.2$ to $x_2 = 0.4$.

Problem 6: Diagnose A Sequential-Update Error. A learner calculates y_{n+1} by Euler and immediately substitutes it into:

$$v_{n+1} = v_n + hg(x_n, y_{n+1}, v_n).$$

Explain why this is not the explicit Euler system update and state the correct formula.

Problem 7: Take One Paired RK4 Step. For:

$$y'' = -y + 2x, \qquad y(0) = 0, \qquad y'(0) = 1,$$

use paired RK4 with $h = 0.2$ to find (y_1, v_1).

Problem 8: Diagnose A Mixed RK4 Stage. A stage-3 calculation uses:

$$\left(x_n + \frac{h}{2}, y_n + \frac{K_2}{2}, v_n + \frac{L_1}{2}\right).$$

Identify the error and write the correct stage-3 state.

Problems 9 To 12: Multistep Methods And Diagnostics

For Problems 9 and 10, use $y'' = x - y$, $h = 0.1$, and this RK4 start-up table:

j	x_j	y_j	$v_j = f_j$	$g_j = x_j - y_j$
0	0.0	1.000000000	0.000000000	−1.000000000
1	0.1	0.995170833	−0.094837500	−0.895170833
2	0.2	0.981397432	−0.178735763	−0.781397432
3	0.3	0.959816580	−0.250856505	−0.659816580

Problem 9: Predict With AB4. Calculate the paired AB4 prediction $(y_4^{(p)}, v_4^{(p)})$ at $x_4 = 0.4$.

Problem 10: Predict With Milne. Use the same table to calculate Milne's paired prediction at $x_4 = 0.4$.

Problem 11: Explain The Start-Up Requirement. Why can a four-step system method not begin from only (y_0, v_0)? State a suitable start-up procedure.

Problem 12: Audit A Damaged History. A proposed ABM4 table has equally spaced x values, but its f_2 was calculated from row 3 and its g_3 was rounded from -0.0048 to 0.00. Identify both risks before the next prediction is accepted.

Worked Answers: Problems 1 To 4

Answer 1. Isolate the highest derivative:

$$y'' = e^x - 3xy' + 2y.$$

Define $v = y'$, so $v' = y''$. Substitute v for y':

$$\boxed{\begin{aligned} y' &= v, \\ v' &= e^x - 3xv + 2y, \\ y(0) &= 2, \\ v(0) &= -1. \end{aligned}}$$

Answer 2. Move the lower-order terms to the right:

$$xy'' = -(1 + x)y' - y.$$

For $x \neq 0$, divide by x:

$$y'' = -\frac{(1 + x)y' + y}{x}.$$

The denominator vanishes at $x = 0$. Therefore this standard-form right-hand side is undefined at the proposed initial point and cannot be passed directly to the displayed numerical formulas.

Answer 3. Define:

$$y_1 = u, \qquad y_2 = u', \qquad y_3 = u''.$$

Solve the original equation for u''':

$$u''' = \cos x - u'' + xu'.$$

Replace the derivatives by state variables:

$$\boxed{\begin{aligned} y_1' &= y_2, \\ y_2' &= y_3, \\ y_3' &= \cos x - y_3 + xy_2. \end{aligned}}$$

Answer 4. Define $v = y'$. Then:

$$f(x, y, v) = v, \qquad g(x, y, v) = x - y - 2v.$$

At $(x_0, y_0, v_0) = (0, 0, 1)$:

$$\begin{aligned} f_0 &= 1, \\ g_0 &= 0 - 0 - 2(1) = -2. \end{aligned}$$

Apply paired Euler:

$$\begin{aligned} y_1 &= 0 + 0.2(1) = 0.2, \\ v_1 &= 1 + 0.2(-2) = 0.6. \end{aligned}$$

The first paired Euler step gives:

$$\boxed{(y_1, v_1) = (0.2, 0.6)}.$$

Worked Answers: Problems 5 To 8

Answer 5. Begin from:

$$(x_1, y_1, v_1) = (0.2, 0.2, 0.6).$$

Calculate both old-state derivatives:

$$\begin{aligned} f_1 &= v_1 = 0.6, \\ g_1 &= x_1 - y_1 - 2v_1 \\ &= 0.2 - 0.2 - 2(0.6) \\ &= -1.2. \end{aligned}$$

Update both components:

$$\begin{aligned} y_2 &= 0.2 + 0.2(0.6) = 0.32, \\ v_2 &= 0.6 + 0.2(-1.2) = 0.36. \end{aligned}$$

The second paired Euler step gives:

$$\boxed{(y_2, v_2) = (0.32, 0.36)}.$$

Answer 6. Explicit Euler evaluates the entire derivative vector at the old state. The correct local formulas are:

$$\boxed{\begin{aligned} y_{n+1} &= y_n + hf(x_n, y_n, v_n), \\ v_{n+1} &= v_n + hg(x_n, y_n, v_n). \end{aligned}}$$

Using y_{n+1} in the second line mixes old and new components and changes the numerical method.

Answer 7. Here:

$$f(x,y,v) = v, \qquad g(x,y,v) = -y + 2x.$$

With $h = 0.2$ and $(x_0, y_0, v_0) = (0,0,1)$, the matched stage pairs are:

$$\begin{aligned}(K_1, L_1) &= (0.2, 0),\\ (K_2, L_2) &= (0.2, 0.02),\\ (K_3, L_3) &= (0.202, 0.02),\\ (K_4, L_4) &= (0.204, 0.0396).\end{aligned}$$

For example, stage 3 uses $(0.1, 0.1, 1.01)$, so:

$$\begin{aligned}K_3 &= 0.2(1.01) = 0.202,\\ L_3 &= 0.2(-0.1 + 2(0.1)) = 0.02.\end{aligned}$$

Combine the K increments:

$$\begin{aligned}y_1 &= 0 + \frac{0.2 + 2(0.2) + 2(0.202) + 0.204}{6}\\ &\approx 0.201333333.\end{aligned}$$

Combine the L increments:

$$\begin{aligned}v_1 &= 1 + \frac{0 + 2(0.02) + 2(0.02) + 0.0396}{6}\\ &\approx 1.019933333.\end{aligned}$$

Combining both RK4 increment sequences gives:

$$\boxed{(y_1, v_1) \approx (0.201333333, 1.019933333)}.$$

Answer 8. Stage 3 must use the complete second-stage pair. The correct state is:

$$\boxed{\left(x_n + \frac{h}{2}, y_n + \frac{K_2}{2}, v_n + \frac{L_2}{2}\right)}.$$

Using L_1 pairs a stage-2 position estimate with a stage-1 velocity estimate.

Worked Answers: Problems 9 To 12

Answer 9. For the f history:

$$\begin{aligned} B_f &= 55f_3 - 59f_2 + 37f_1 - 9f_0 \\ &\approx -6.760685308. \end{aligned}$$

Substitute the f bracket into the AB4 position predictor:

$$\begin{aligned} y_4^{(p)} &= y_3 + \frac{0.1}{24} B_f \\ &= 0.959816580 + \frac{0.1}{24}(-6.760685308) \\ &\approx 0.931647058. \end{aligned}$$

For the g history:

$$\begin{aligned} B_g &= 55g_3 - 59g_2 + 37g_1 - 9g_0 \\ &\approx -14.308784266. \end{aligned}$$

Substitute the g bracket into the AB4 velocity predictor:

$$\begin{aligned} v_4^{(p)} &= v_3 + \frac{0.1}{24} B_g \\ &= -0.250856505 + \frac{0.1}{24}(-14.308784266) \\ &\approx -0.310476440. \end{aligned}$$

The paired AB4 prediction is:

$$\boxed{(y_4^{(p)}, v_4^{(p)}) \approx (0.931647058, -0.310476440)}.$$

Answer 10. For Milne's position predictor:

$$\begin{aligned} M_f &= 2f_3 - f_2 + 2f_1 \\ &\approx -0.512652248, \\ y_4^{(p)} &= y_0 + \frac{4(0.1)}{3} M_f \\ &\approx 0.931646367. \end{aligned}$$

For Milne's velocity predictor:

$$\begin{aligned} M_g &= 2g_3 - g_2 + 2g_1 \\ &\approx -2.328577395, \\ v_4^{(p)} &= v_0 + \frac{4(0.1)}{3} M_g \\ &\approx -0.310476986. \end{aligned}$$

The paired Milne prediction is:

$$\boxed{(y_4^{(p)}, v_4^{(p)}) \approx (0.931646367, -0.310476986)}.$$

Answer 11. A four-step formula needs four complete state rows and their derivative rows:

$$(y_0, v_0), \quad (y_1, v_1), \quad (y_2, v_2), \quad (y_3, v_3),$$

together with $f_0, \dots, f_3$ and $g_0, \dots, g_3$.

The initial conditions provide only the first row. Use RK4 with the intended step size to generate rows 1, 2, and 3, evaluate both derivatives at each row, and then begin the multistep method with $n = 3$.

Answer 12. The value f_2 must be calculated from row 2:

$$f_2 = f(x_2, y_2, v_2).$$

Using row 3 shifts the history and attaches the wrong time index to that derivative.

Also, rounding $g_3 = -0.0048$ to zero destroys all significant information in that slope. Multistep coefficients can magnify the resulting error. Recompute f_2 from the correct row and retain sufficient precision in g_3 before predicting.

Chapter 20 Final Summary

Reduce:

$$y'' = F(x, y, y')$$

by defining $v = y'$:

$$\boxed{y' = v, \qquad v' = F(x, y, v)}.$$

Euler advances both old-state derivatives together:

$$\boxed{\begin{aligned} y_{n+1} &= y_n + hf_n, \\ v_{n+1} &= v_n + hg_n. \end{aligned}}$$

RK4 builds matched stage pairs (K_j, L_j) and combines each component with the weights $1, 2, 2, 1$.

ABM4 and Milne require four complete RK4-generated state rows. For either method:

preserve two derivative histories, predict both state components, evaluate both right-hand sides at the same predicted endpoint, and correct each component with its matching history.

The central lesson is:

a higher-order scalar equation becomes a larger state, and every numerical stage must keep that entire state synchronised.